Major Project
Bug Hunting on Open Bug Bounty Target

Dramane Traore

Abstract

This project demonstrates the practical application of bug hunting methodology on a deliberately vulnerable web application (OWASP Juice Shop), simulating the real-world process used on Open Bug Bounty targets. The project involved systematic reconnaissance, vulnerability scanning, manual testing, and exploitation to identify security weaknesses, including Cross-Site Scripting (XSS) and SQL Injection vulnerabilities. Two critical vulnerabilities were successfully identified, exploited, and documented, following the full disclosure lifecycle from discovery to reporting. This hands-on exercise reflects the same workflow a security researcher would follow when participating in a coordinated disclosure program such as Open Bug Bounty, while operating in a legally safe and fully controlled environment.

Introduction

The rapid growth of digital technologies has transformed the way individuals, businesses, and governments operate, communicate, and exchange data. This digital transformation, however, has significantly expanded the attack surface available to malicious actors, increasing the risk of data breaches, unauthorized access, and financial loss. Web applications in particular remain one of the most targeted attack vectors, as they are often publicly accessible and process large volumes of user-supplied input.

Platforms such as Open Bug Bounty play a critical role in improving web security at scale by connecting independent security researchers with website owners through a coordinated vulnerability disclosure process. Rather than requiring a formal bug bounty contract, the platform allows researchers to report vulnerabilities they discover on any public website, which are then verified and relayed to the site owner.

This project applies that same testing philosophy and methodology to OWASP Juice Shop, an intentionally vulnerable web application maintained by OWASP (Open Web Application Security Project) and widely used in security training, certification programs, and academic coursework. Using Juice Shop as a safe practice target allowed for a full, unrestricted exploitation cycle — something not always possible on

live production targets — while still reflecting the exact categories of vulnerabilities most frequently found in real-world bug bounty submissions (XSS, injection flaws, and broken authentication).

Tools and Environment

- Target application: OWASP Juice Shop (Node.js / Express / Angular stack)
- Deployment method: Docker container (bkimminich/juice-shop), run locally on macOS (Apple M1)
- Access point: http://localhost:3000
- Testing approach: Manual black-box testing via browser
- Browser used for testing: Safari (macOS)

Running the target locally via Docker ensured complete isolation from any production system, eliminating any legal or ethical concerns typically associated with testing third-party infrastructure without explicit authorization.

Testing Methodology

1. Reconnaissance: The target application was deployed locally and explored manually to map its functionality and identify user-facing input points. Key areas identified included the product search bar, the login form, and the user account/basket system.
2. Scanning: Given the time constraints of the exercise, automated scanning tools were deprioritized in favor of targeted manual testing of the highest-value input points (search and authentication).
3. Manual testing: Each identified input field was tested with crafted payloads designed to probe for a lack of input sanitization (XSS) and unsafe query construction (SQL Injection). Testing was iterative: an initial basic payload was filtered by the application, prompting the use of a more advanced payload that successfully bypassed the filtering mechanism.
4. Fuzzing: Limited manual fuzzing was performed on the login form by testing common SQL Injection authentication-bypass patterns until a successful payload was identified.
5. Exploitation: Two vulnerabilities were successfully exploited — a DOM-based XSS and an authentication bypass via SQL Injection — each confirmed with visual proof of impact.

6. Reporting: Each vulnerability was documented following a standard disclosure format: location, payload, root cause, impact, severity, and proof of concept.
7. Verification: Both vulnerabilities were independently reproduced twice to confirm consistency and rule out false positives.
8. Documentation: All testing steps, payloads, and results were logged in real time, including failed attempts, to maintain an accurate and transparent testing record.

Findings

Vulnerability 1 — DOM-Based Cross-Site Scripting (XSS)

Location	Product search field (/#/search?q=...)
Payload	<img src=x onerror=alert('XSS')>
Vulnerability class	CWE-79 (Improper Neutralization of Input During Web Page Generation)
Severity	Medium–High

Description

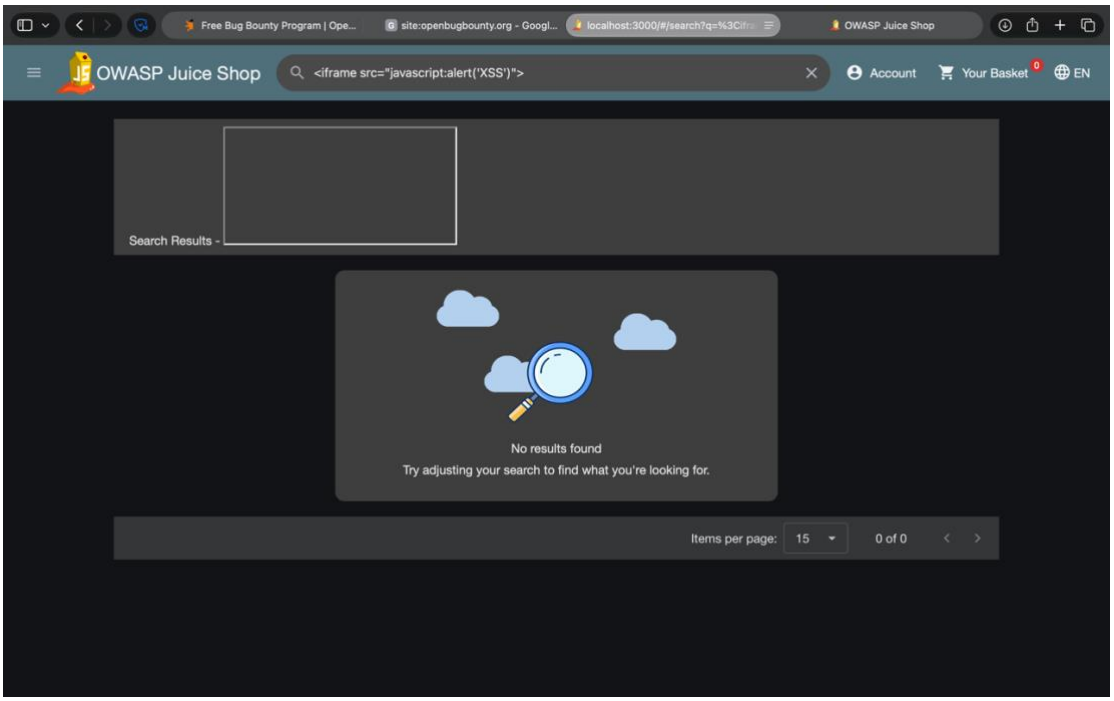
The search parameter q is reflected into the application's DOM without proper output encoding or sanitization. An initial test using a standard <script> tag was blocked by the application's input filtering, indicating partial — but insufficient — protection. A secondary payload using an <img> tag with an invalid source and an onerror event handler successfully bypassed this filtering, executing arbitrary JavaScript in the browser.

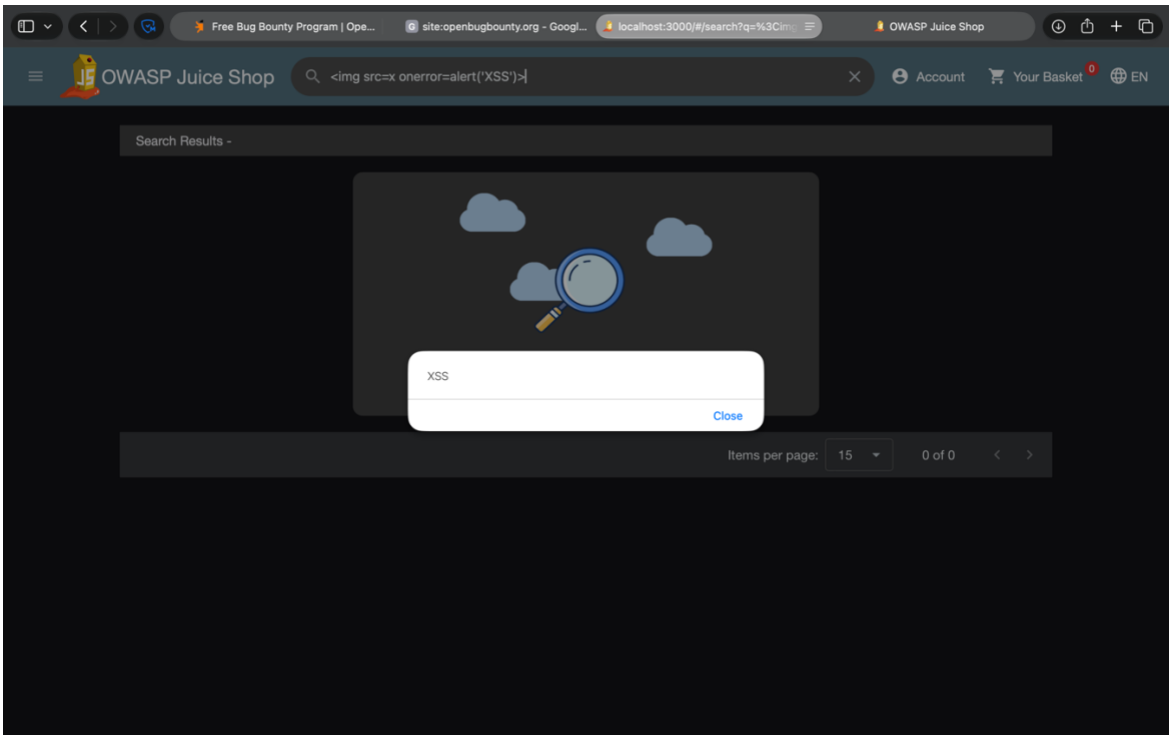
Impact

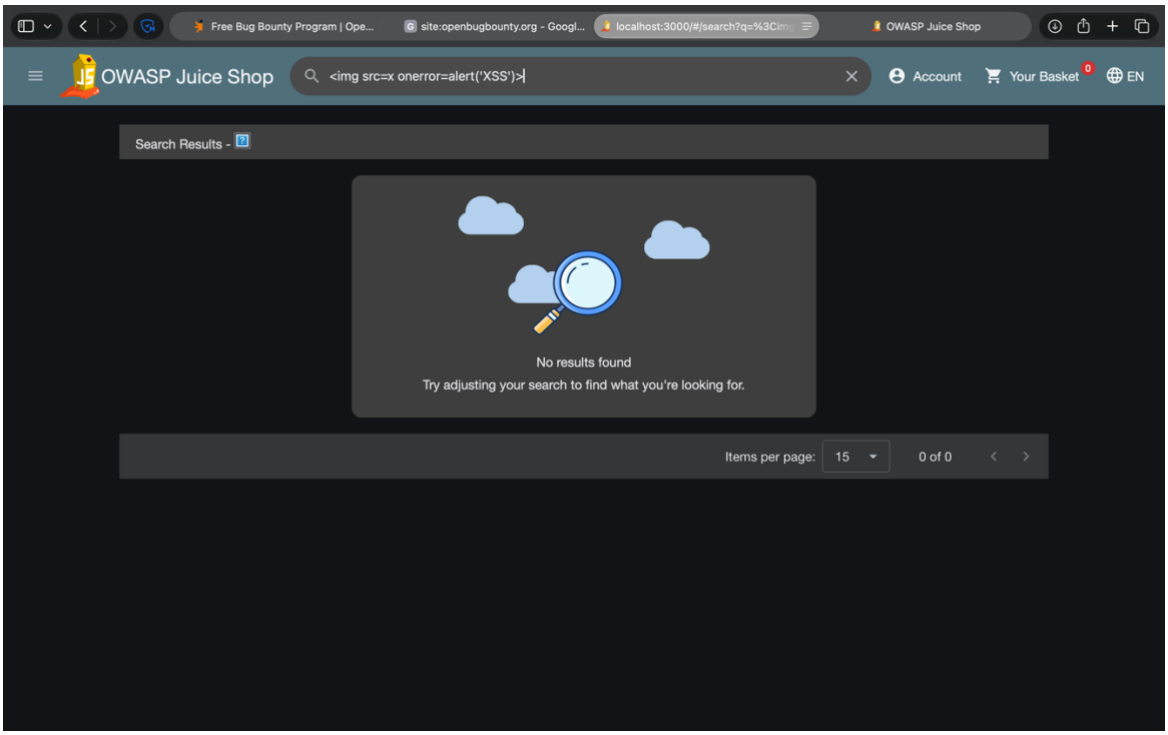
In a real-world scenario, this vulnerability could be leveraged by an attacker to hijack user sessions, steal cookies or authentication tokens, redirect victims to phishing pages, or perform actions on behalf of the victim without their consent (session riding).

Proof of Concept

The payload was submitted via the search bar; the resulting page rendered a JavaScript alert() dialog box displaying "XSS", confirming successful script execution in the DOM context.


Free Bug Bounty Program | Ope...
site:openbugbounty.org - Googl...
localhost:3000/#/search?q=%3Cifra
OWASP Juice Shop
OWASP Juice Shop
<iframe src="javascript:alert('XSS')">
Account
Your Basket
0
EN
Search Results -
No results found
Try adjusting your search to find what you're looking for.
Items per page:
15
0 of 0


Free Bug Bounty Program | Ope...
site:openbugbounty.org - Googl...
localhost:3000/#/search?q=%3Cimg
OWASP Juice Shop
OWASP Juice Shop
<img src=x onerror=alert('XSS')>
Account
Your Basket
0
EN
Search Results -
XSS
Close
Items per page:
15
0 of 0



Vulnerability 2 — SQL Injection (Authentication Bypass)

Location	Login form (/#/login), email field
Payload	' OR 1=1--
Vulnerability class	CWE-89 (SQL Injection)
Severity	Critical

Description

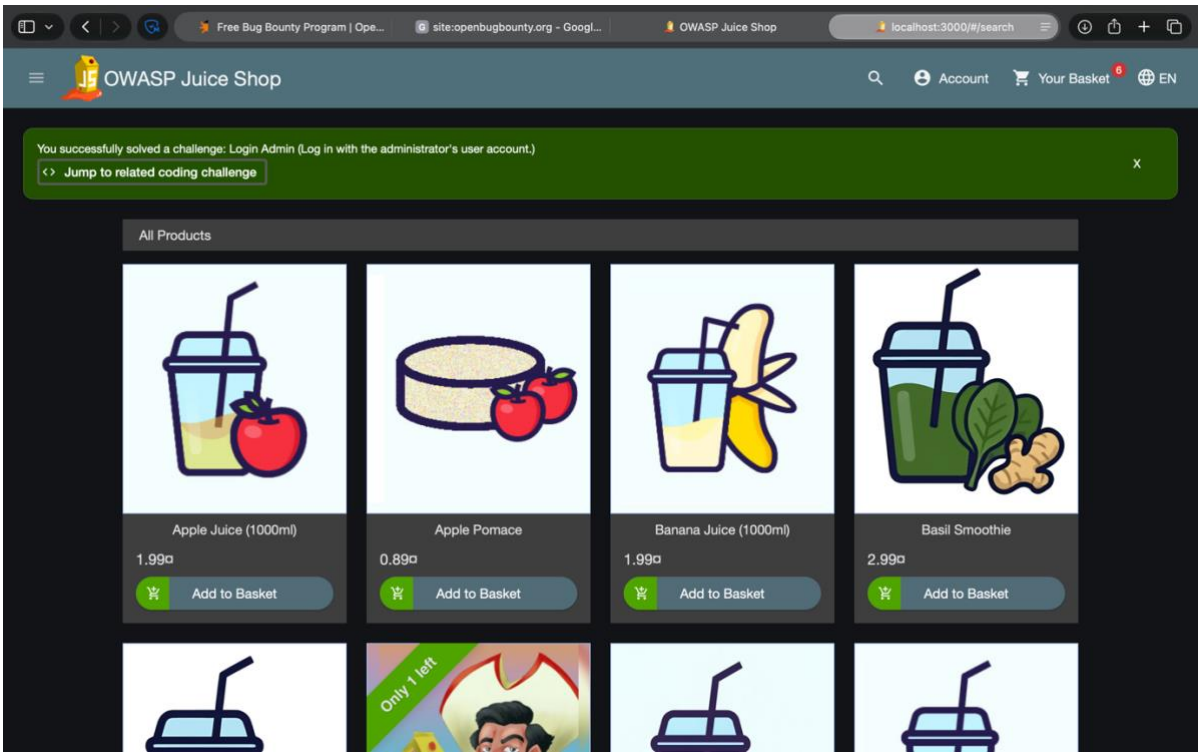
The backend authentication query concatenates the user-supplied email field directly into a SQL statement without using parameterized queries or prepared statements. Submitting the payload ' OR 1=1-- manipulates the query's WHERE clause so that its condition always evaluates to true, causing the database to return the first matching user record — in this case, the administrator account — regardless of the password supplied.

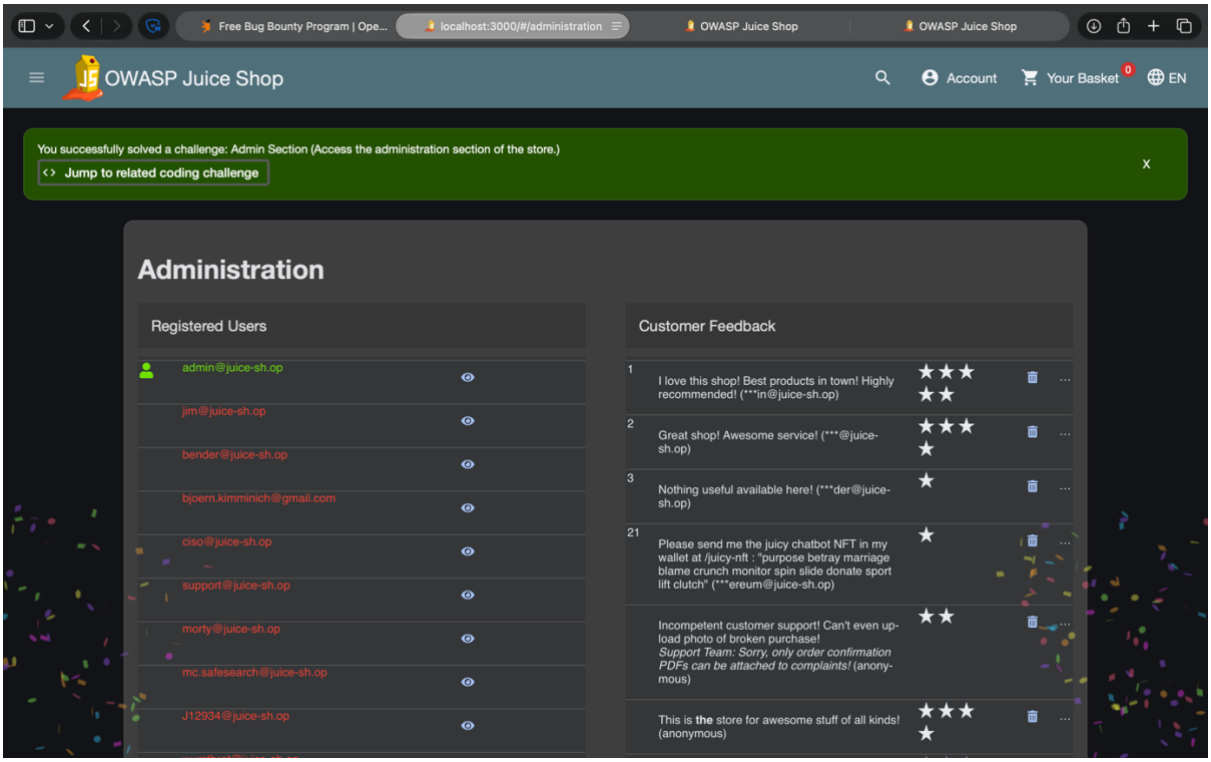
Impact

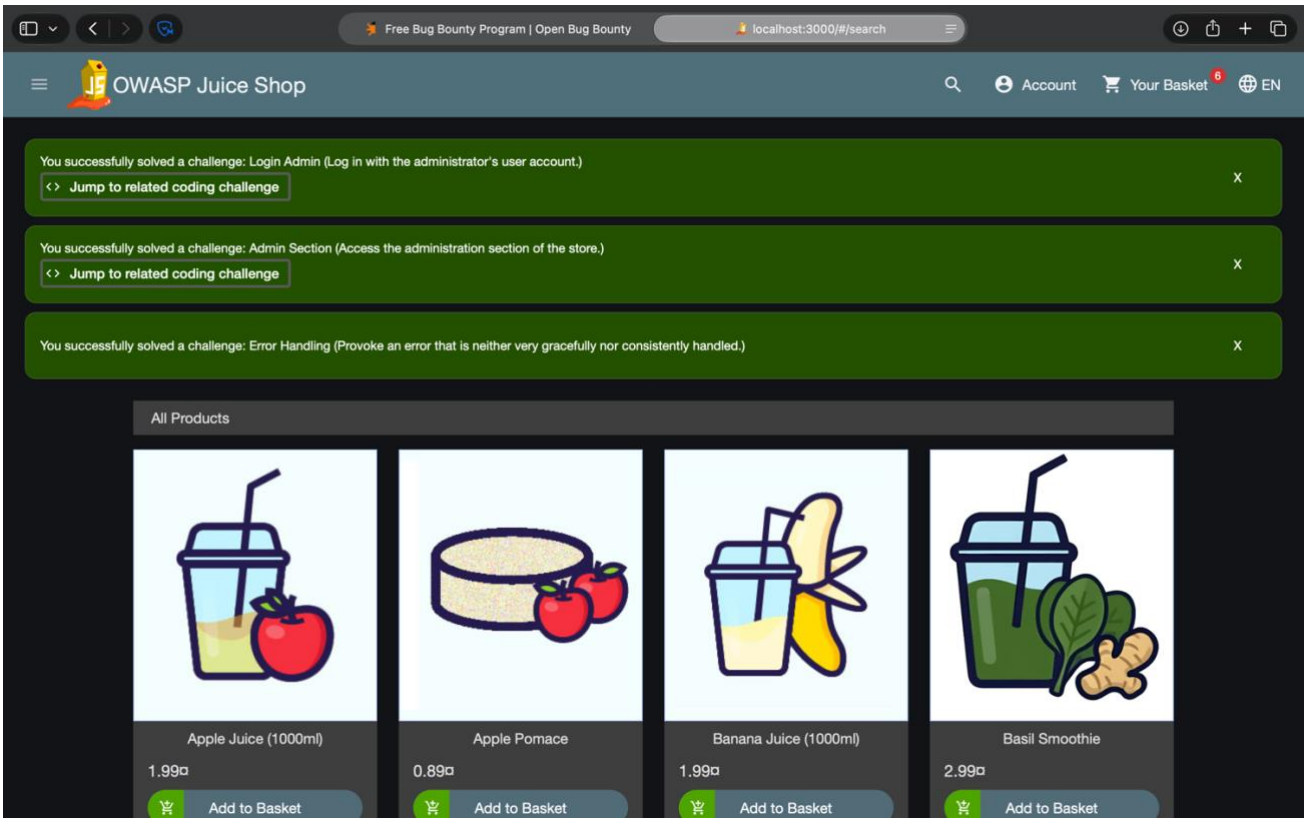
This vulnerability results in a complete authentication bypass, allowing an unauthenticated attacker to gain administrator-level access to the application without any valid credentials. This is the most severe class of vulnerability identified in this assessment, as it compromises the integrity of the entire authentication system.

Proof of Concept

Submitting the payload in the email field (with an arbitrary password) resulted in successful login as the administrator account, independently confirmed by the application's built-in challenge-tracking system, which flagged the "Login Admin" challenge as solved.







Additional Testing (Negative Result)

An Insecure Direct Object Reference (IDOR) test was attempted against the basket API (/rest/basket/{id}) by manually iterating basket IDs while authenticated. This test did not succeed, as the endpoint correctly enforced authentication via a required Authorization header (JWT), returning a 401 Unauthorized error for all unauthenticated requests. This negative result is documented for completeness and demonstrates that not all tested endpoints were vulnerable — reinforcing that the application implements inconsistent, rather than entirely absent, security controls.

Conclusion

This assessment identified two significant vulnerabilities within OWASP Juice Shop: a DOM-based Cross-Site Scripting flaw and a critical SQL Injection vulnerability enabling full authentication bypass. Both findings represent common, high-impact vulnerability classes that frequently appear in real-world bug bounty reports on the Open Bug Bounty platform. The contrast between the successful XSS/SQLi exploitation and the unsuccessful IDOR attempt illustrates that security is rarely binary across an application — some areas may be well protected (such as the JWT-secured basket API) while others remain critically exposed (such as the login mechanism). This reinforces the importance of comprehensive, consistent security

testing across every component of a web application rather than assuming uniform protection.

Remediation Recommendations

- Implement proper contextual output encoding/escaping for all user-supplied input reflected into the DOM, particularly within search and display components.
- Replace all dynamic SQL query construction with parameterized queries or prepared statements across the entire authentication layer.
- Apply a strict Content Security Policy (CSP) to reduce the impact of any residual or future XSS vectors.
- Enforce consistent authentication and authorization checks across all API endpoints, following the model already correctly implemented on the basket API.
- Integrate automated static and dynamic security testing (SAST/DAST) into the CI/CD pipeline to catch similar issues before deployment.
- Conduct periodic manual penetration testing to complement automated tooling, as demonstrated by the manual discovery of both vulnerabilities in this report.